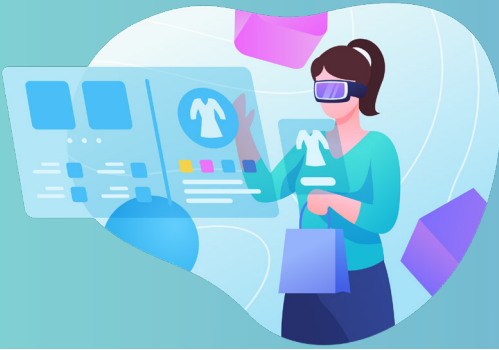
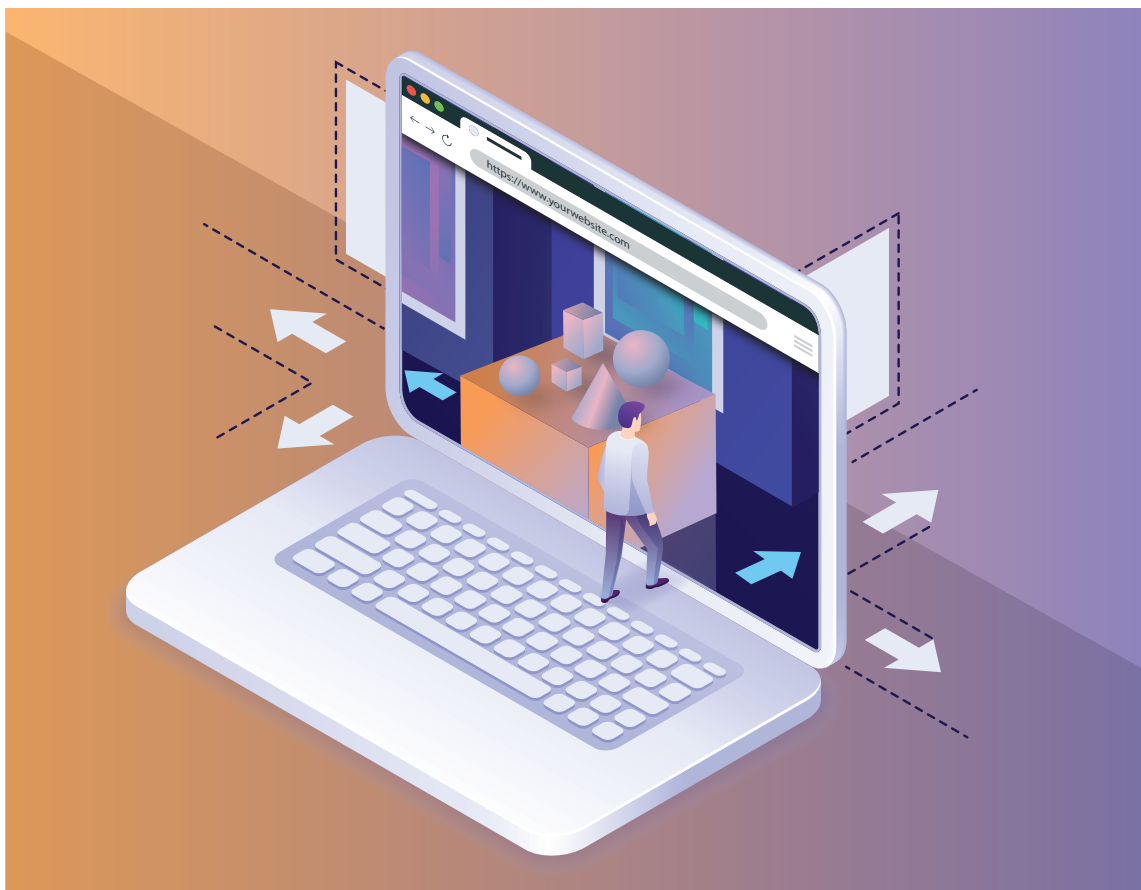


Introduction to Three.js



01

Three.js helps developers create interactive and visually stunning 3D environments directly within a web browser, enhancing user experiences with immersive and dynamic content.



In the field of interactive media design, Three.js emerges as a powerful library that seamlessly blends 3D modelling with web development. It allows us to create dynamic and interactive three-dimensional environments that come to life directly in the web browser.

Previously, we explored 3D modelling and web development as separate topics. Now, we will bring these concepts together by diving into how three.js merges these fields. This exploration will enable you to create immersive and engaging web experiences.

These two disciplines—3D modelling and web development—are distinct yet increasingly interconnected. As digital experiences evolve, the boundaries between these areas continue to blur, resulting in richer and more immersive online environments.

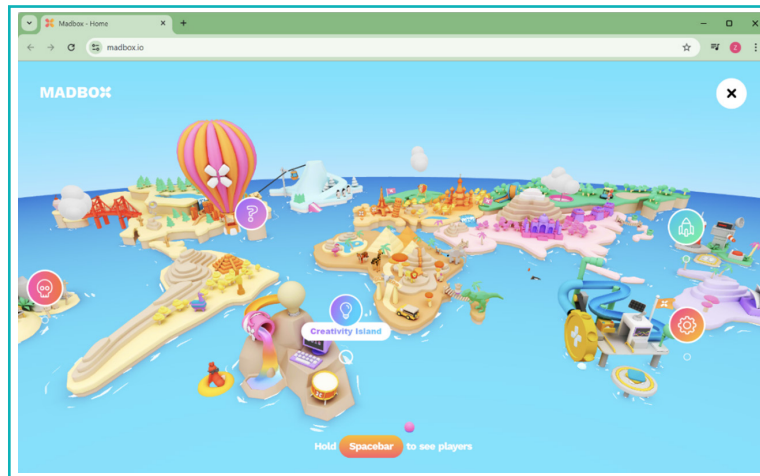


Image credits: madbox.io

With advancements in web technology, it is now possible to embed 3D models directly into web pages. This allows users to interact with 3D content in real-time, rotating objects, zooming in, or exploring virtual environments without leaving their browsers.

Web development increasingly uses 3D simulations for education, training, and entertainment, enabling users to experience scenarios that would be difficult or impossible in real life. Examples include virtual labs, architectural walkthroughs, or product demos.

Also, web designers are beginning to explore how 3D elements can enhance user interfaces, making them more engaging and intuitive. For example, 3D buttons, interactive maps, or product views can provide a richer experience than traditional 2D elements.

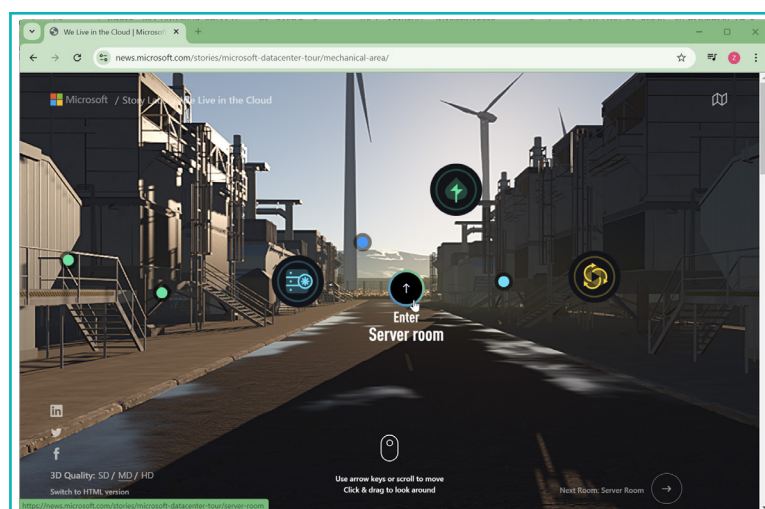
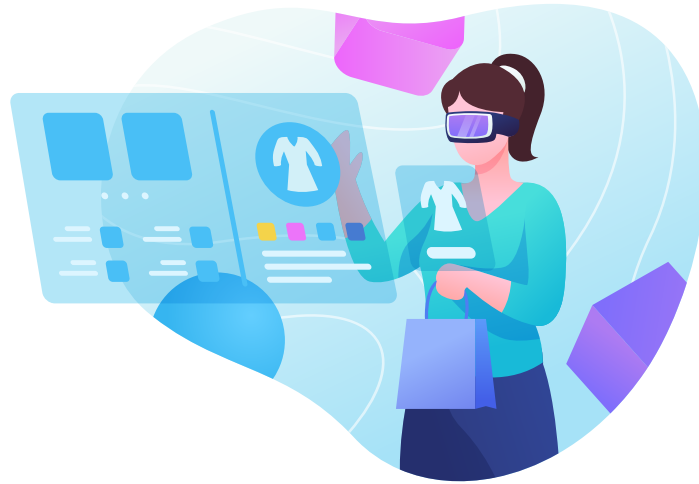
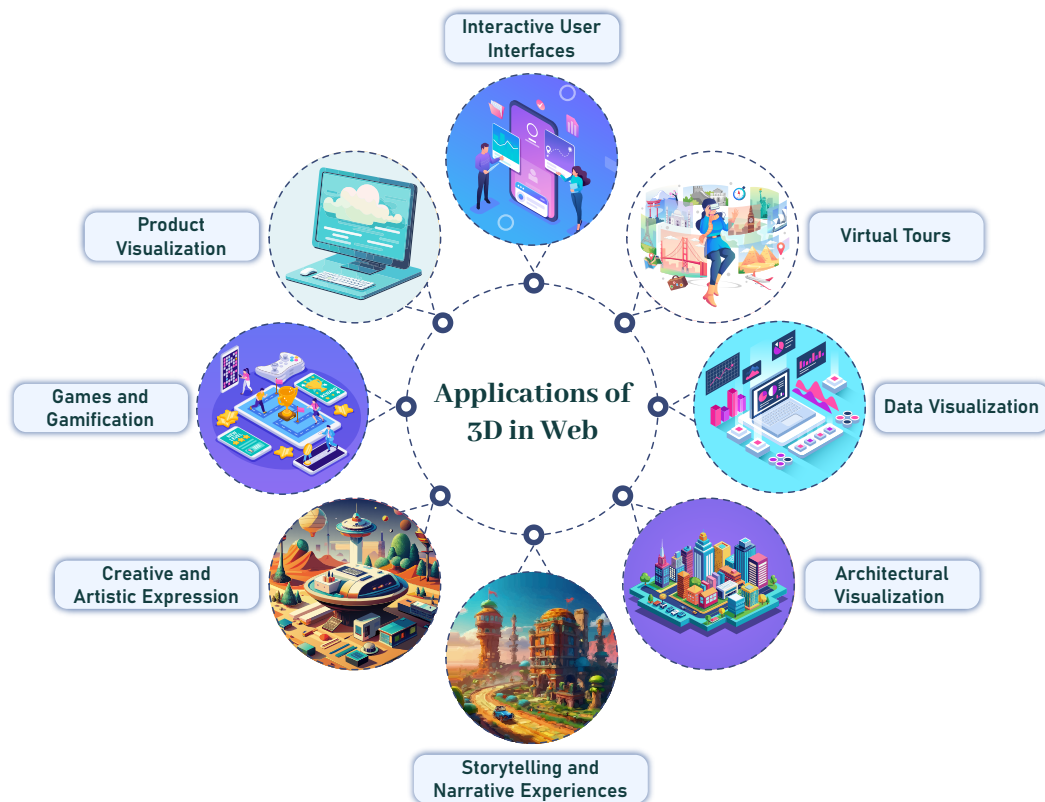


Image credits: Gizmos Chamber

The rise of VR and AR pushes web development towards incorporating 3D design principles. Web-based VR/AR experiences require a deep understanding of both 3D design (to create the content) and web development (to deliver it effectively across different devices).



Applications of 3D in Web Design



Creating 3D Web Experiences

Achieving 3D web development involves combining various techniques and technologies to create immersive three-dimensional experiences on the web.

A general approach to 3D web development can be outlined as follows:

- Get acquainted with the fundamentals of 3D graphics by learning to create 3D models using software such as Blender and Maya, and understanding the basics of rendering 3D scenes.
- Choose the right tools and libraries for creating 3D graphics in the web browser.
- Set up your development environment with essential technologies such as HTML, CSS, and JavaScript.
- Create and load 3D models.
- Set up the 3D scene, including the camera, lighting, and objects.
- Implement the interactive features and animations to enhance the user experience.
- Ensure your application runs smoothly by optimising assets and code.
- Test your application across different devices and browsers before deploying it to a web server.

Getting started

When creating an HTML webpage, we usually save it as an HTML file on our computer and open it by double-clicking it. For simple and static content without external dependencies, this method works well and poses minimal security risks. However, when an HTML webpage relies on external resources, like WebGL textures, browsers enforce security restrictions on files when opened directly. These restrictions can limit the ability to test complex features.

While developing three.js websites, we need to load 3D models, textures, or other external assets. As a result, testing the page locally is not feasible without a server. To address this, we must set up and run a local server.

Recall:

A **Server** is like a central computer that provides information and services to other computers requesting it.

Local Server

A Local Server operates on your personal computer or within a local network rather than being hosted on the internet or in a remote data centre. Using a local server is more secure and practical for web development, since it closely mimics the production environment, enabling thorough testing of web application features. Setting up a local server requires some additional configuration, often involving the use of a 'build tool' or 'bundler'.

Bundler

A Bundler is a tool used in web development to compile JavaScript code and its dependencies into a single file (or a few files), making it easy to include in a web page. This process is known as **Bundling**.

There are several build tools available today, with some of the most popular ones being Webpack, Vite, Gulp, and Parcel. For our projects, we will use **Vite**, which is considered an excellent build tool for modern web development.

Vite has gained popularity for its speed, simplicity, and modern features. It installs and runs quickly and is less prone to bugs. Vite is suitable for both small projects and large applications due to its efficient build process. It compiles HTML, CSS, and JavaScript files into a cohesive final website.

Activity

We will create a webpage featuring a basic 3D shape. This shape will be positioned within a scene that can be explored in three dimensions, similar to interacting with objects in a 3D modelling environment.

This introductory project will help you understand the core concepts of 3D graphics and scene setup in a browser.

Part 1 – Initial Setup

For coding our three.js project, we will use the Visual Studio Code (VSCode) editor. If it is not already installed, ensure to download and set it up.

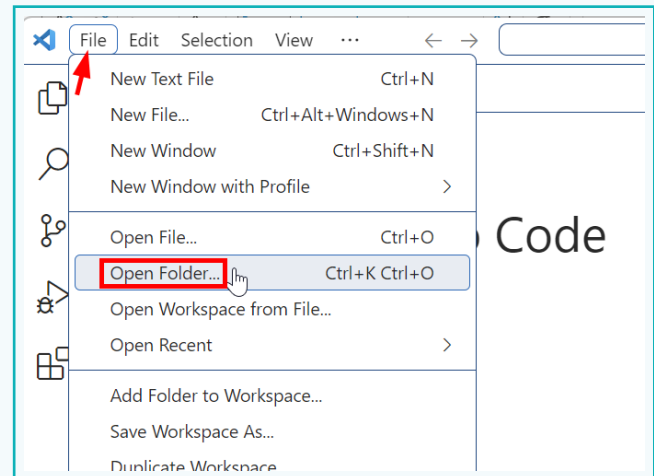
Note:

To run the build tool (Vite) and our three.js websites, make sure that node.js is installed on the computer. Node.js allows JavaScript to execute outside the browser and is necessary for running tools such as Vite.

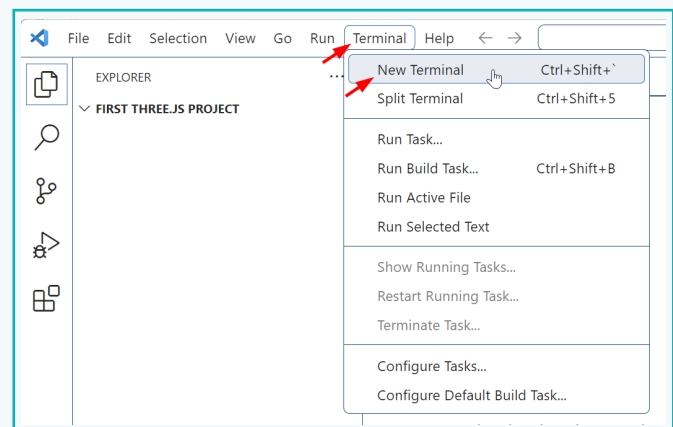


Activity

- 1 Once node.js has been installed, create a folder that will contain your website. Launch VSCode and open the created folder in the editor.



- 2 Open a new terminal by navigating to the 'Terminal' menu and selecting 'New Terminal,' or use the keyboard shortcut 'CTRL + J.'



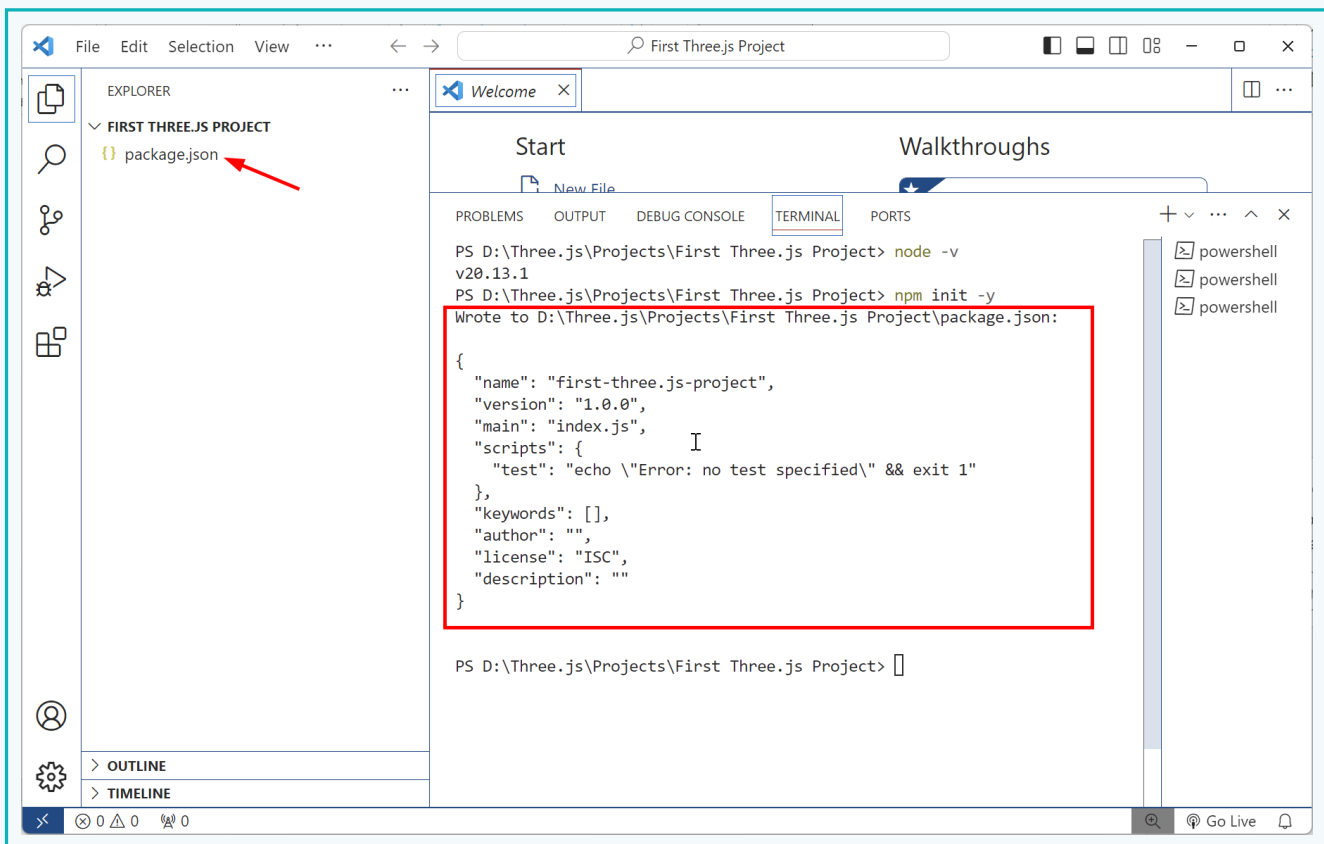
- Run `npm init -y` on the terminal.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Three.js\Projects\First Three.js Project> node -v
v20.13.1
PS D:\Three.js\Projects\First Three.js Project> npm init -y
```

Activity

Running `npm init -y` generates a 'package.json' file in our project folder, containing the basic information required to set up a Node.js project.



The screenshot shows the Visual Studio Code interface for a project named 'First Three.js Project'. In the Explorer sidebar on the left, a red arrow points to the 'package.json' file under the 'FIRST THREE.JS PROJECT' folder. The main editor area displays the 'Start' and 'Walkthroughs' tabs. The 'TERMINAL' tab is active, showing the following commands and output:

```
PS D:\Three.js\Projects\First Three.js Project> node -v
v20.13.1
PS D:\Three.js\Projects\First Three.js Project> npm init -y
Wrote to D:\Three.js\Projects\First Three.js Project\package.json:

{
  "name": "first-three.js-project",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}
```

The output of the `npm init -y` command is highlighted with a red box. The terminal prompt is now `PS D:\Three.js\Projects\First Three.js Project>`.

Recall:

- When Node.js is installed, the **Node Package Manager (NPM)** is also installed by default. NPM is a dependency manager that helps install, update, and manage packages (like three.js, Vite, and other libraries) for JavaScript projects.
- Additionally, NPM provides a Command Line Interface (CLI) called **npm**, which allows for installing, updating, and removing packages, managing package versions, running scripts defined in the 'package.json' file, and handling project dependencies.

Activity

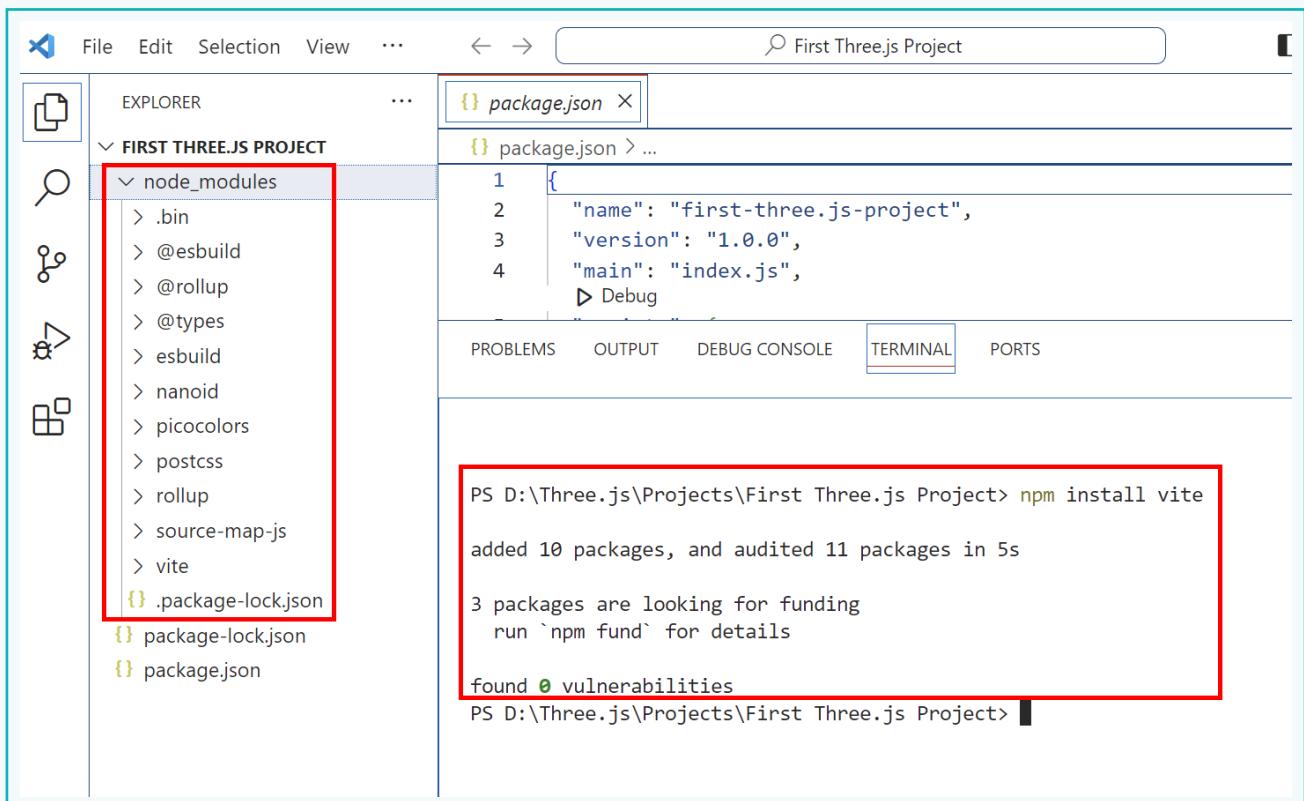
Part 2 - Add Vite Dependency

Our project will utilise two dependencies— Vite and three.js.

- 1 Add the 'Vite' dependency to the Node.js project by running the command - `npm install vite` in the terminal.

```
PS D:\Three.js\Projects\First Three.js Project> npm install vite
```

The command `npm install vite` creates a 'node_modules' folder, which contains all the dependencies required for the project. It is important to note that the contents of this folder should not be manually altered. In addition to the 'vite' dependency itself, the folder also includes other dependencies that vite relies on.



The screenshot shows the Visual Studio Code interface for a project named 'First Three.js Project'. In the Explorer on the left, the 'node_modules' folder is expanded, showing a list of installed dependencies including .bin, @esbuild, @rollup, @types, esbuild, nanoid, picocolors, postcss, rollup, source-map-js, vite, .package-lock.json, package-lock.json, and package.json. The vite dependency is highlighted with a red box. The terminal at the bottom shows the command `npm install vite` being executed, with the output: 'added 10 packages, and audited 11 packages in 5s', '3 packages are looking for funding', 'run `npm fund` for details', and 'found 0 vulnerabilities'. The terminal output is also highlighted with a red box.

Activity

The 'package.json' file is updated to include a list of dependencies under the 'dependencies' section. For instance, if Vite is installed, it will be added there along with its version number, which may allow for flexibility with compatible versions.

```
{} package.json X
{} package.json > ...
1  {
2    "name": "first-three.js-project",
3    "version": "1.0.0",
4    "main": "index.js",
5    "scripts": {
6      "test": "echo \\\"Error: no test specified\\\" && exit 1"
7    },
8    "keywords": [],
9    "author": "",
10   "license": "ISC",
11   "description": "",
12   "dependencies": {
13     "vite": "^5.4.2"
14   }
15 }
16
```

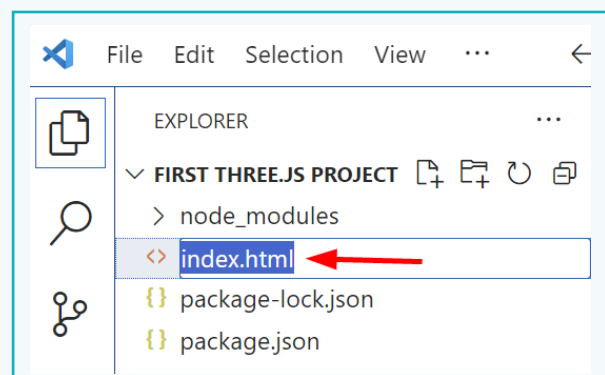
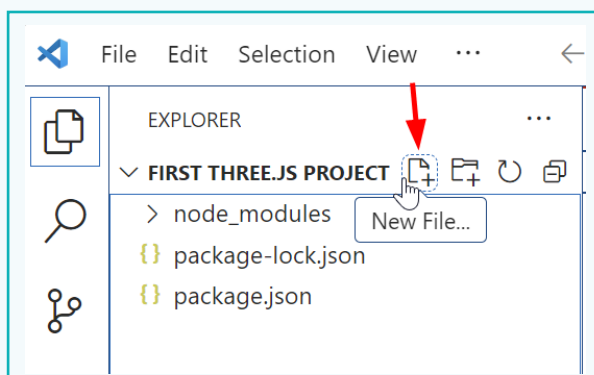
Note:

The 'package-lock.json' file records the exact versions of all dependencies installed in your project. Together with the 'package.json' file, this allows you to share your project. To share your three.js website with another developer, delete the 'node_modules' folder and share the rest. The other developer can then run a command to rebuild the 'node_modules' folder based on the dependencies listed in 'package.json'.

Part 3 - Basic Website

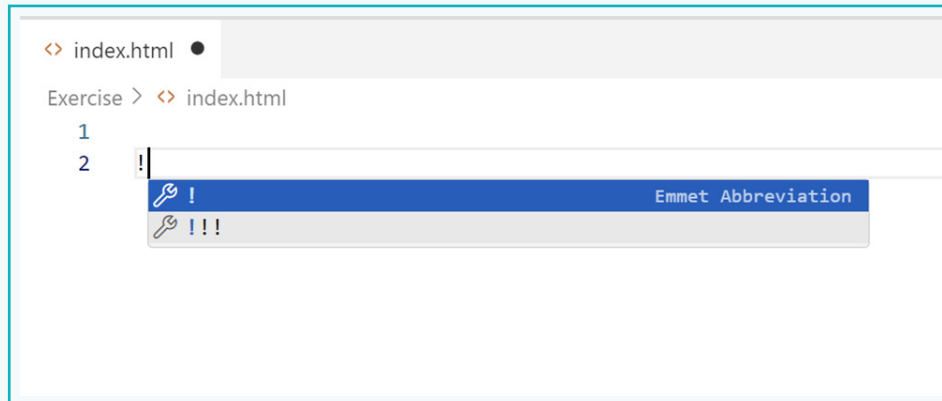
Now, we will start creating a basic website.

- 1 Create an 'index.html' file in the same folder.



Activity

- Type '!' to insert an HTML boilerplate code.



```
<> index.html •
Exercise > <> index.html
1
2 !
   ! Emmet Abbreviation
   !!!
```

HTML boilerplate:

An HTML boilerplate is a pre-prepared template of code that serves as a starting point for web projects. It saves time by providing a foundational structure, allowing developers to avoid starting from scratch with each new project.

- 1 Change the title and add a `<h1>` tag within the body tag.



```
<> index.html X
Exercise > <> index.html > ...
1
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title> First Three.js project </title>
8 </head>
9 <body>
10  <h1> Three.js website - Under Construction </h1>
11 </body>
12 </html>
13
```

Before we start coding, an additional step is required. Within the 'package.json' file, locate the 'scripts' field. This section defines custom commands that can be executed using npm (Node Package Manager).

Activity

These scripts automate repetitive tasks like starting a server, running tests, building your project, and more. The 'scripts' field is a key-value pair that represents a script name and its associated command.

```
json
{
  "scripts": {
    "script-name": "command"
  }
}
```

Here,

script-name is an identifier you choose for your script.

command is the actual shell command or series of commands to be executed.

We can place the *vite* command in the 'scripts' field of the 'package.json' file, since this will allow us to easily run the command.

Note:

The *vite* command is used to start a local development server.

- 2 In the 'package.json' file, modify the value within the 'scripts' field as shown.



```
Exercise > {} package.json > ...
1 {
2   "name": "exercise",
3   "version": "1.0.0",
4   "main": "index.js",
5   "scripts": {
6     "test": "echo \"Error: no test specified\" && exit 1"
7   },
8   "keywords": [],
9   "author": "",
10  "license": "ISC",
11  "description": "",
12  "dependencies": {
13    "three": "^0.164.1",
14    "vite": "^5.2.11"
15  }
16 }
17
```

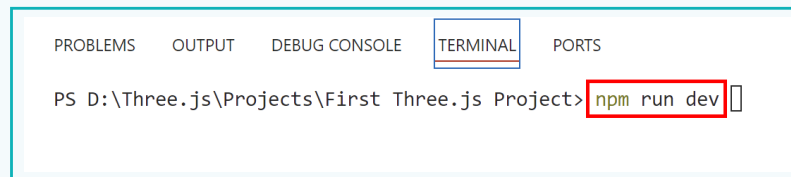


```
{} package.json > {} scripts > dev
1 {
2   "name": "first-three.js-project",
3   "version": "1.0.0",
4   "main": "index.js",
5   "scripts": {
6     "dev": "vite"
7   },
8   "keywords": [],
9   "author": "",
10  "license": "ISC",
11  "description": "",
12  "dependencies": {
13    "vite": "^5.4.2"
14  }
15 }
```

Activity

Test it by running the dev script in the terminal.

- Run `npm run dev`.



```
PS D:\Three.js\Projects\First Three.js Project> npm run dev
```

Running this command will open a page in your default browser. The URL will be of the form: <http://localhost:5173/>, where 5173 is the default port for Vite.



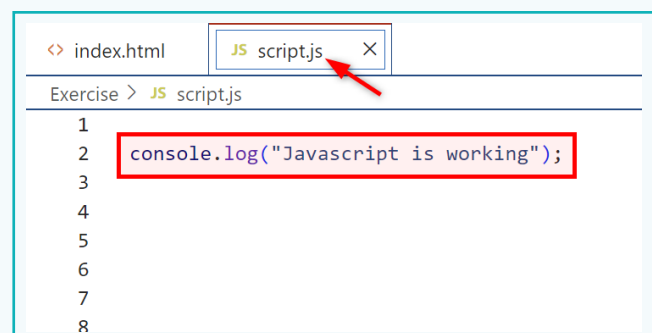
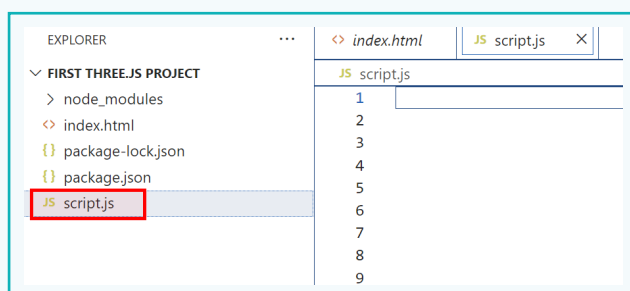
Bytes

A **default port** functions as a standardised channel through which a computer directs and receives specific types of information .

With the basic HTML website in place, the next step is to add JavaScript code.

Part 4 - Add JavaScript Code

- 1 Create a 'script.js' file. Add a line of code to check if JavaScript is working or not.



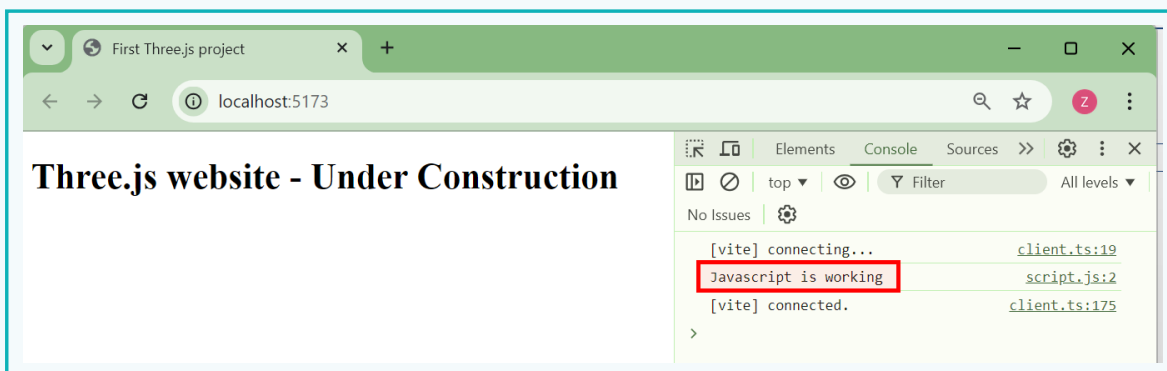
Activity

- Also, add the marked line of code to the 'index.html' file.



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title> First Three.js project </title>
7 </head>
8 <body>
9   <h1> Three.js website - Under Construction </h1>
10  <script type="module" src="./script.js"></script>
11 </body>
12 </html>
```

- Go back to the browser and open the developer tools (Shift + Ctrl + J).



Note:

The page automatically reloads without manual intervention because Vite detects changes and updates the page in real time.

Activity

Part 5 - Add the three Dependency

Now that basic HTML and JavaScript are working, it is time to add three.js code. To do so, we must first install the 'three' dependency into our project.

The name of the dependency on NPM is 'three'. Just as we installed the Vite dependency, we can install the 'three' dependency as shown below—

npm install three

However, if you observe the terminal, it is currently busy running the 'Vite' server.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

VITE v5.2.11 ready in 387 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
█
```

There are two options:

- Open a new terminal and run the command, ensuring you are in the same directory.
- Stop the server, run the command, and then restart the server.

We will proceed with the second option by stopping the server first.

- 1 Press 'CTRL + C' to stop the server. On Windows, you may need to press the shortcut multiple times or confirm the action by typing 'Y'.

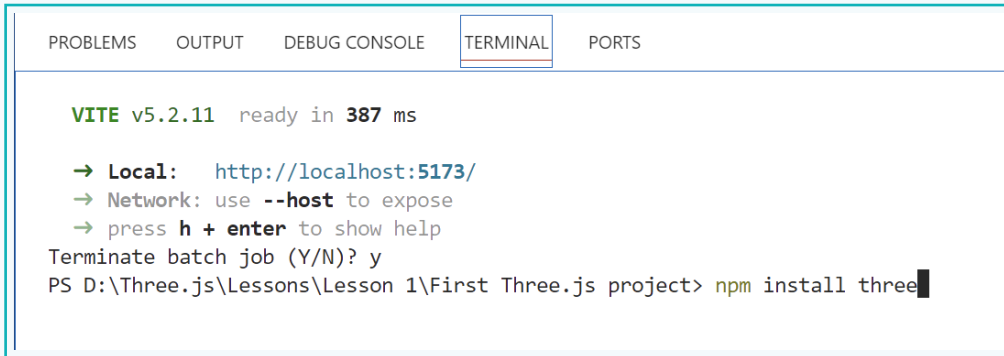
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

VITE v5.2.11 ready in 387 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
Terminate batch job (Y/N)? y█
```

Activity

- 2 Now, run `npm install three`.

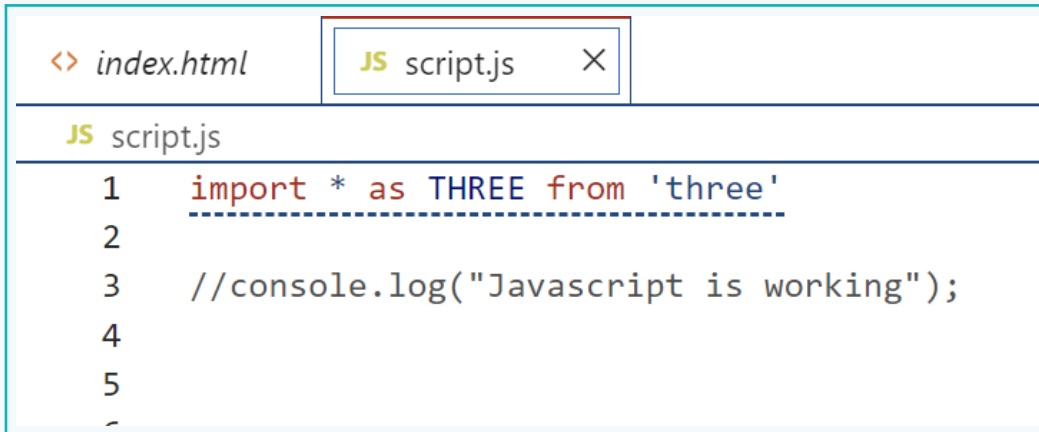


```
VITE v5.2.11 ready in 387 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
Terminate batch job (Y/N)? y
PS D:\Three.js\Lessons\Lesson 1\First Three.js project> npm install three
```

To utilise the properties and methods of the 'three' package, it must be imported into the project.

- 3 In 'script.js', import the 'three' dependency into the project.



```
<> index.html  JS script.js  X

JS script.js
1  import * as THREE from 'three'
2
3  //console.log("Javascript is working");
4
5
~
```

This will import all core classes from the *three* package into the *THREE* variable.

Note:

The name of this variable must be written in uppercase.

Activity

- 4 Write a console statement to check the contents of this variable.

```
<> index.html JS script.js X
JS script.js
1 import * as THREE from 'three'
2
3 console.log(THREE);
4
5
6
```

The *THREE* variable contains most of the classes and properties required for a three.js project. Let us learn how to access them and create a basic three.js scene.

Part 6 - Create the First three.js Scene

To create a three.js scene, we need four elements:

- A. Scene
- B. Objects
- C. Camera
- D. Renderer

Let us explore how to create each of these elements.

A. Scene

The Scene is like a container. You place your objects, models, particles, lights, etc., in it. When done, you ask three.js to render that scene.

- 1 Create a scene using the *Scene* class.

Note:

To use any of the classes within the *three* package, it must be instantiated as shown.

```
<> index.html JS script.js X
JS script.js > ...
1 import * as THREE from 'three'
2
3 //console.log(THREE);
4
5
6 //Scene
7 const scene = new THREE.Scene();
8
9
10
```

Activity

B. Objects

Objects in 3D can include a variety of elements such as primitive geometries, imported models, particles, and lights. We will start with a simple red cube. For any basic shape, a Mesh— a type of 3D object— must first be defined.

Mesh

Mesh is a combination of **geometry** (the shape) and **material** (how it looks).

There are many geometries available in three.js such as –

- **BoxGeometry**: A 3D box (cuboid)
- **SphereGeometry**: A sphere or ball
- **CylinderGeometry**: A cylinder or tube
- **ConeGeometry**: A cone shape

And many more

Some of the materials available in three.js are –

- **MeshBasicMaterial**: A simple material with a solid colour or texture that is not affected by lighting.
- **MeshLambertMaterial**: A material that reacts to lighting and has a matte finish.
- **MeshPhongMaterial**: A material with a shiny finish that reflects light and can show specular highlights.
- **MeshStandardMaterial**: A physically based material that simulates real-world lighting interactions.

And many more

Refer to the documentation for a comprehensive list of available geometries and materials. <https://threejs.org/docs/index.html#manual/en>



- 2 To create the geometry of the cube, we must use the *BoxGeometry* class to create an instance. Provide three parameters that determine the box's dimensions.

```
<> index.html JS script.js X
JS script.js > ...
1 import * as THREE from 'three'
2
3 //console.log(THREE);
4
5
6 //Scene
7 const scene = new THREE.Scene();
8
9
10 //Object
11 const geometry = new THREE.BoxGeometry(1,1,1);
12
13
```

Activity

- 3 To create the material, use the *MeshBasicMaterial* class with one parameter. This parameter is an object {} containing a few options. We will just specify the *color* property.

There are many ways to specify a colour in three.js -

- JS hexadecimal like '0xff0000'
- String hexadecimal like '#ff0000'
- Colour names like 'red'

```
6 //Scene
7 const scene = new THREE.Scene();
8
9
10 //Object
11 const geometry = new THREE.BoxGeometry(1,1,1);
12
13 const material = new THREE.MeshBasicMaterial(
14   {
15     color: 'red'
16   }
17 )
18
19
```

- 4 To create the final mesh, we use the *Mesh* class and send the *geometry* and the *material* as parameters.

```
10 //Object
11 const geometry = new THREE.BoxGeometry(1,1,1);
12
13 const material = new THREE.MeshBasicMaterial(
14   {
15     color: 'red'
16   }
17 )
18
19 const mesh = new THREE.Mesh(geometry, material);
20
```

- 5 Add the mesh to the scene by using the *add()* method.

```
10 //Object
11 const geometry = new THREE.BoxGeometry(1,1,1);
12
13 const material = new THREE.MeshBasicMaterial(
14   {
15     color: 'red'
16   }
17 )
18
19 const mesh = new THREE.Mesh(geometry, material);
20
21 scene.add(mesh);
22
```

Activity

Note:

An object must be added to the scene; otherwise, it will not be visible when rendered.

C. Camera

The Camera itself is not visible within the scene. The rendering shows the scene from the camera's viewpoint, much like how a camera captures an image.

To create the camera, we use the *PerspectiveCamera* class. This class mimics the way our eyes perceive the world, giving a sense of depth and perspective, thus creating realistic 3D scenes. It accepts four parameters, two of which are mandatory.

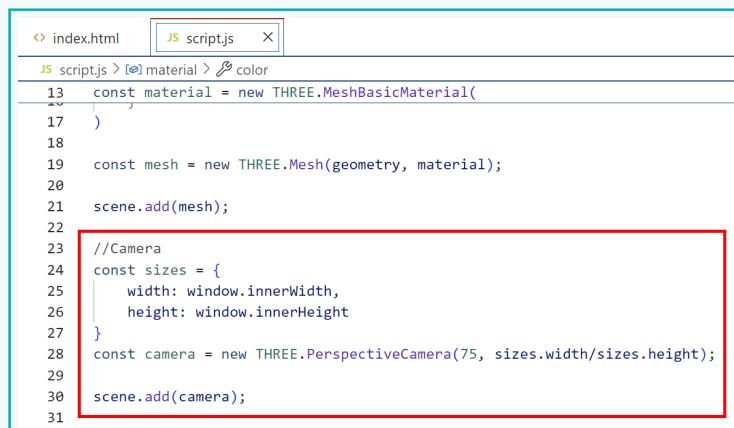
- i. **Field of View:** Defines the extent of the scene visible to the camera, measured in degrees. A larger value results in a wider view. For our purposes, let us set the field of view as '75'.
- ii. **Aspect ratio:** Defines the aspect ratio of the camera, typically set as the width of the viewport divided by its height. This ensures that the scene is not distorted when rendered.

To manage the aspect ratio, we create an object named "sizes" to store the browser window's dimensions. *window.innerWidth* and *window.innerHeight* provide the current width and height of the viewport (the visible area of the browser window). Using the *sizes* object simplifies our code by avoiding repetitive use of these properties.

6 Add the line of code for creating the *sizes* object.

- Also, add a line to create a new *PerspectiveCamera* instance using 'three.js'. Provide the *field of view* and aspect *ratio* values as parameters.

Finally, add the *camera* to the scene.



```
<> index.html JS script.js X
JS script.js > [material] > [color]
13 const material = new THREE.MeshBasicMaterial(
17 )
18
19 const mesh = new THREE.Mesh(geometry, material);
20
21 scene.add(mesh);
22
23 //Camera
24 const sizes = {
25   width: window.innerWidth,
26   height: window.innerHeight
27 }
28 const camera = new THREE.PerspectiveCamera(75, sizes.width/sizes.height);
29
30 scene.add(camera);
31
```

Activity

D. Renderer

Finally, we will render the scene. The renderer will render our scene from the camera's viewpoint, drawing the result onto the canvas.

Firstly, we will create an HTML `<canvas>` element; the canvas is where three.js will render the 3D scene.

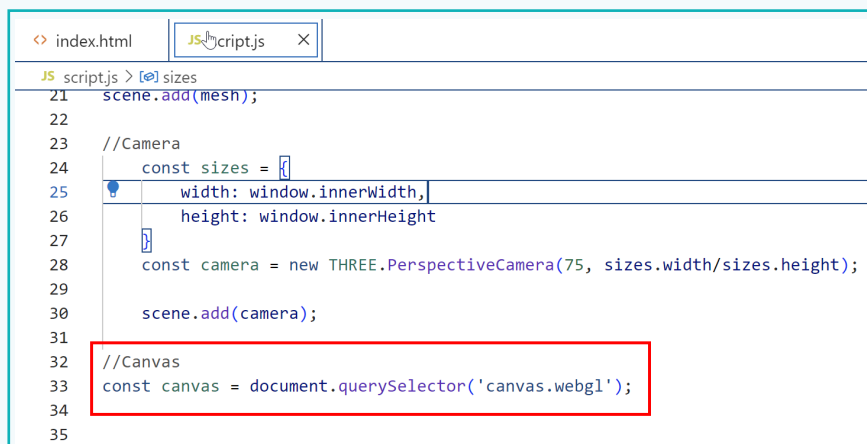
- 7 In 'index.html', instead of the `<h1>`, create the `<canvas>` element before loading the script. Provide it with a class name.



```
1
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title> First Three.js project </title>
8 </head>
9 <body>
10   <!-- <h1> Three.js website - Under Construction </h1> -->
11   <canvas class="webgl"></canvas>
12
13   <script type="module" src="./script.js"></script>
14
15
16 </body>
17 </html>
18
```

The `class="webgl"` attribute allows us to easily select this specific canvas in JavaScript using the `querySelector` method. It also helps with styling if needed.

- 8 Next, select the canvas in script.js using the `document.querySelector()` method. Assign the canvas to a variable, as it can be used for other purposes later.



```
21 scene.add(mesh);
22
23 //Camera
24 const sizes = {
25   width: window.innerWidth,
26   height: window.innerHeight
27 }
28 const camera = new THREE.PerspectiveCamera(75, sizes.width/sizes.height);
29
30 scene.add(camera);
31
32 //Canvas
33 const canvas = document.querySelector('canvas.webgl');
34
35
```

Activity

Note:

The *querySelector* method is used to find the first element in the document that matches the specified CSS selector (in this case, *canvas.webgl*).

Now, to create the renderer, we will use the *WebGLRenderer* class of *three.js*, with one parameter—an object `{}` containing all the options. We must specify the *canvas* property corresponding to the `<canvas>` element we added to the page.

- 9 Create a new *WebGLRenderer* instance. This will be responsible for rendering the 3D scene. Then, pass *canvas* as a parameter to instruct *three.js* to render the scene onto the existing *canvas* element.

```
28   const camera = new THREE.PerspectiveCamera(75, sizes.width/sizes.height);
29
30   scene.add(camera);
31
32   //Canvas
33   const canvas = document.querySelector('canvas.webgl');
34
35   //Renderer
36   const renderer = new THREE.WebGLRenderer({
37     canvas: canvas
38   });
39
40
```

Note:

WebGLRenderer is the default renderer in *three.js*, which uses *WebGL* to render the scene. **WebGL** is a JavaScript API that enables rendering 3D graphics directly in the browser without the need for plugins.

We also need to update the size of the renderer using the *setSize()* method. For this, use the *sizes* object created earlier.

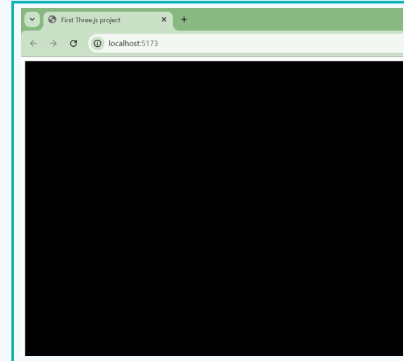
- 10 Call the *setSize()* method and pass the *width* and *height* properties of the *sizes* object.

```
31
32   //Canvas
33   const canvas = document.querySelector('canvas.webgl');
34
35   //Renderer
36   const renderer = new THREE.WebGLRenderer({
37     canvas: canvas
38   });
39
40   renderer.setSize(sizes.width, sizes.height);
41
42
43
44
```

Activity

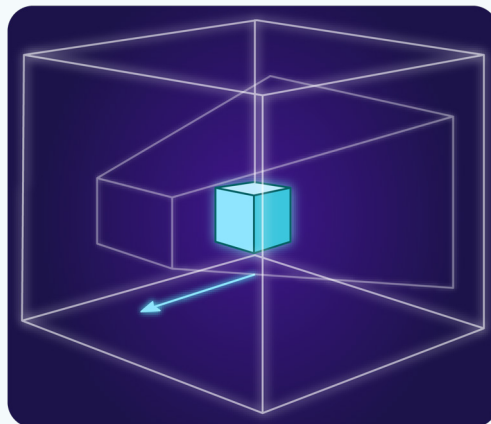
- 11 Finally, call the `render()` method on the renderer. Pass the `scene` and the `camera` as parameters.

```
32 //Canvas
33 const canvas = document.querySelector('canvas.webgl');
34
35
36 //Renderer
37 const renderer = new THREE.WebGLRenderer({
38   canvas: canvas
39 })
40
41 renderer.setSize(sizes.width, sizes.height);
42
43 renderer.render(scene, camera);
44
45
```



The output is not as expected, and only a blank screen is visible.

This is because the camera, which allows viewing the scene from different angles, is positioned at the same default location as the cube. In other words, the camera is most probably inside the cube, hence, nothing is visible on the screen.



To solve this problem, let us move the camera backwards.

We will use the properties such as position, rotation and scale. The `position` property has three values, `x`, `y` and `z`. By default, three.js considers the forward/backward axis to be `z`.

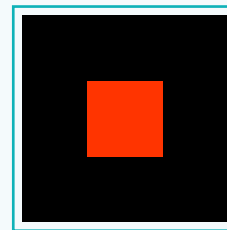
Note:

We will learn the 'transform' operations in detail in the next chapter.

Activity

- 12 Provide a positive value to the *z* value of the *position* property of the camera. This statement must be present after the camera has been created and before the render.

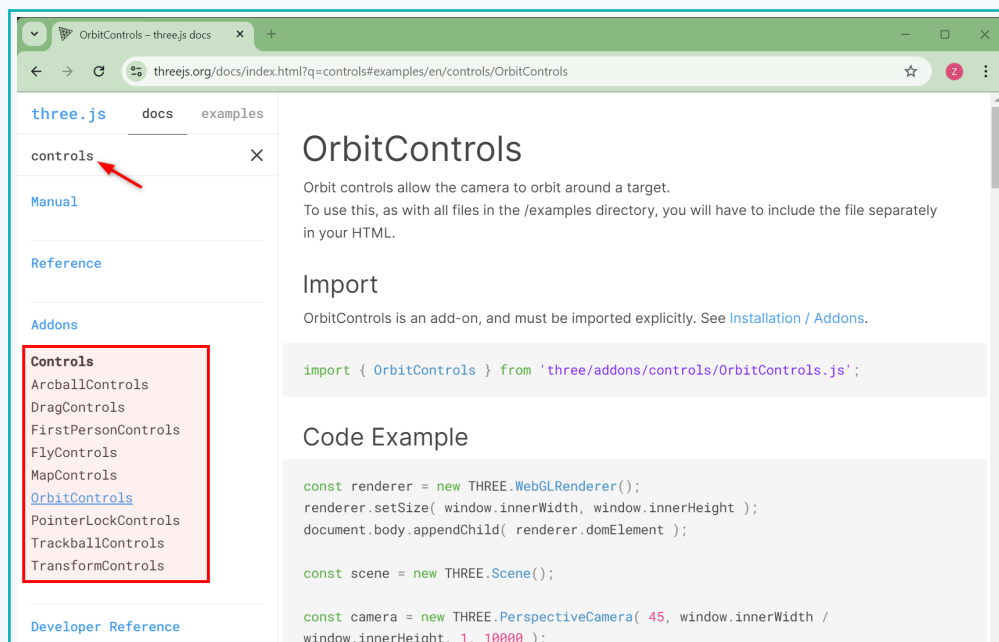
```
23 //Sizes
24 const sizes = {
25   width: 800,
26   height: 600
27 }
28
29 const camera = new THREE.PerspectiveCamera(75, sizes.width/sizes.height);
30
31 camera.position.z = 3
32
33 scene.add(camera);
34
```



The first render of the three.js website is visible. The cube looks like a square because the camera is aligned perfectly with the cube. Only one side is visible to us.

However, this output is not very interesting. It would be much more exciting if we could navigate the scene just like in the viewport of any 3D modelling software. Let us add a few more lines of code to understand the magic of three.js.

three.js offers various classes, allowing us to control the camera and much more. You can explore them by typing 'controls' in the three.js documentation.



To use these classes in the program, the respective library or class must first be imported. In this case, the required class is 'OrbitControls'.

Activity

- 13 Import the class *OrbitControls* as:

import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls.js'

```
<> index.html JS script.js X
JS script.js > ...
1 import * as THREE from 'three'
2
3 import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls.js'
4
5 //Scene
6 const scene = new THREE.Scene();
7
8
```

- 14 Create an instance of the *OrbitControls* class to be able to use it in our code.

```
28 const camera = new THREE.PerspectiveCamera(75, sizes.width/sizes.height);
29 camera.position.z = 3;
30 scene.add(camera);
31
32 //Canvas
33 const canvas = document.querySelector('canvas.webgl');
34
35 const controls = new OrbitControls(camera, canvas);
36
37 //Renderer
38 const renderer = new THREE.WebGLRenderer({
39   canvas: canvas
40 })
```

- 15 Comment out the existing *renderer.render()* method. This statement will need to be moved into a function that will get executed in a loop.

Write the function *animate()*. Copy the render statement into it. Also, write the *requestAnimationFrame()* method as shown in the image below.

```
<> index.html JS script.js X
JS script.js > ...
39
40 renderer.setSize(sizes.width, sizes.height);
41 // renderer.render(scene, camera); ← Comment
42
43
44 function animate()
45 {
46   renderer.render(scene, camera);
47   window.requestAnimationFrame(animate);
48 }
49
50 animate() ← Call the function
51
```

Activity

Note:

We will not dive deeply into the above lines of code right now. We will simply observe the output. Feel free to explore it further on your own until we examine it in detail in the next lesson.

Output -

Go back to the browser and explore the scene to view all sides of the 3D cube, similar to how you would navigate in the viewport of any 3D modelling software like Blender or Maya.



Congratulations! You have just created your first 3D webpage. In the next lesson, we will dive deeper into this topic and include some exciting features in our project.

In this lesson, we learned the basics of Three.js and 3D web development. We successfully developed a webpage incorporating a 3D object that allows user interaction, demonstrating how 3D elements can enhance user experience and engagement in web development.

Exercise

Fill in the Blanks.

- 1 While developing three.js websites, we must set up and run a _____.
- 2 A _____ is a tool used in web development to compile JavaScript code and its dependencies into a single file (or a few files).
- 3 Running `npm init -y` generates a _____ file in our project folder.
- 4 _____ is a dependency manager that helps install, update, and manage packages.
- 5 The _____ command is used to start a local development server.
- 6 _____ defines the extent of the scene visible to the camera.

State if the statements are True or False.

- 7 A local server gets hosted on the internet or in a remote data centre.
 _____
- 8 To create the geometry of a cube, we must use the `SphereGeometry` class.
 _____
- 9 It is necessary to add an object to the scene for it to be visible.
 _____
- 10 There is only one way to specify a colour in three.js.
 _____
- 11 The renderer renders the scene from the camera's viewpoint.
 _____

Tech Flash

- Local server** : It is a server that operates on your personal computer or within a local network.
- Bundler** : It is a tool used in web development to compile JavaScript code and its dependencies into a single file (or a few files) that can be easily included in a web page.
- NPM** : It is a dependency manager that helps install, update, and manage packages.
- Field of View** : It defines the extent of the scene visible to the camera, measured in degrees.
- Aspect ratio** : It defines the aspect ratio of the camera, typically set as the width of the viewport divided by its height.